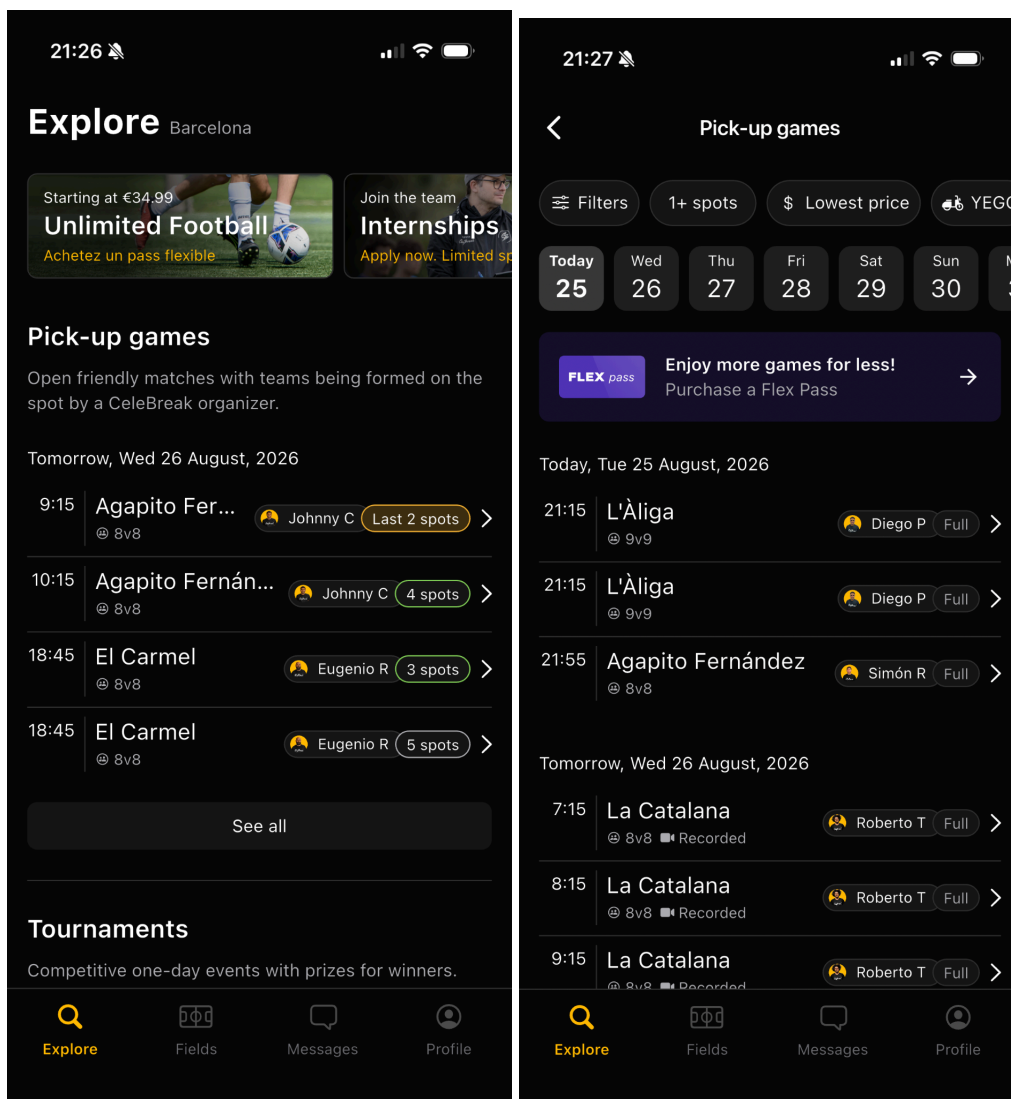


Technical Task: Explore Screens + Games Q&A Agent

This task has two parts built around the same feature: discovering pick-up games.

- **Part 1. Screen implementation:** Replicate the two Explore screens from the provided screenshots as closely as possible.



- **Part 2. Games Q&A agent:** Expose the games data as an API an AI agent can consume reliably, and build a minimal chat agent that answers questions about the games in natural language.

Both parts share the same data: the screens and the agent should read from the same small local API (a mock data file behind it is fine, no real database needed).

Please implement your solution using Flutter (v3.44.8) + GraphQL/Node.

Part 1: Explore Screens

Design reference: the two attached screenshots above.

Screen 1: Explore home

- "Explore" title with the city name ("Barcelona") next to it.
- Horizontally scrolling promo banner carousel: image cards with rounded corners, a text overlay (small heading, large bold title, yellow call-to-action line).
OPTIONAL.
- "Pick-up games" section: title, short description text.
- A date header (e.g. "Tomorrow, Wed 26 August, 2026") followed by a short list of game rows (see the game row spec below).
- "See all" button navigating to Screen 2.
- Start of a "Tournaments" section (title + description is enough; its list is out of scope).
- Bottom navigation bar: Explore (active, highlighted in yellow), Fields, Messages, Profile. Other tabs may be placeholders.

Screen 2: Pick-up games list

- Back arrow and centered "Pick-up games" title.
- Horizontally scrolling filter chip row: "Filters", "1+ spots", "Lowest price", "YEGO" (only "1+ spots" needs to actually work, it hides full games; the others may be inert).
- Horizontally scrolling date selector ("Today 25", "Wed 26", ...) with a visually distinct selected state. Selecting a date scrolls/filters the list to that day.
- Flex Pass promo banner: badge, two-line text, arrow.
- Games grouped under date headers ("Today, Tue 25 August, 2026", "Tomorrow, ..."), using the same game row component as Screen 1.
- Bottom navigation bar as on Screen 1.

Game row component (shared between both screens)

- Start time on the left, separated from the content by a thin vertical divider.

- Venue name, truncated with an ellipsis when too long.
- Format line with an icon (e.g. "8v8", "9v9").
- Organizer chip: small circular avatar + first name and initial (e.g. "Johnny C").
- Availability chip, color-coded by state:
 - Urgent (few spots left): yellow/amber filled, e.g. "Last 2 spots".
 - Available: green outline, e.g. "4 spots".
 - Full: neutral gray, "Full".
- Chevron on the right; the whole row is tappable (a placeholder detail screen is fine).

Part 2: Games Q&A Agent

We are making our app usable by AI agents, so our APIs must be unambiguous and safe when consumed by a language model. Design the games API with that in mind, then prove it works by driving it from a minimal chat agent that answers users' questions about the games.

2a. The API

Implement a small local server (GraphQL) that serves the games shown in the screenshots (10–20 mock games across several days is plenty):

- Query games with filters: date or date range, venue, minimum spots available.
- Fetch one game's details.

2b. The agent

Build a minimal chat agent (Simple UI, no polish needed) using any LLM API (Please use a free option: Google Gemini API recommended), with your games API exposed as tools.

The agent must handle at least these conversations correctly:

1. **Simple question:** "What games are there tomorrow morning?" - agent queries the API with the right date and answers using only returned data, presenting times, venues, and spots accurately.
2. **Relative dates:** "Tomorrow", "this weekend", "on Thursday" resolve to correct concrete dates (state your reference "today" in the mock data).

3. **Grounded answers only:** The agent never invents games, venues, times, or availability. If the API returns nothing for a query, it says so plainly.
4. **Ambiguity:** "Any spots at La Catalana?" when games exist on several days - agent asks which day or clearly breaks the answer down per day, rather than answering for an arbitrary one.
5. **Honest about limits:** Questions the API cannot answer (e.g. "which game has the best players?") get an honest answer rather than a fabricated answer.

Deliverables

1. Repository with the app screens, the API, and the agent.
2. README.md covering:
 - How to run each part.
 - Your API design decisions.
 - What you cut or would do differently with more time.